

SOFTWARE DEVELOPMENT LIFE CYCLE

Engineering & Delivery Blueprint

How AuraFit AI is planned, built, tested, and shipped as a privacy-first local desktop application.

What's Inside

01 Purpose & Methodology

Why a lightweight Agile-Kanban hybrid fits a local-first, AI-assisted desktop product.

02 Lifecycle Phases

The seven stages AuraFit AI moves through from concept to maintenance.

03 Roles & Responsibilities

Functional "hats" required, mappable to a solo builder or a small team.

04 Environments & Branching

Git strategy, build channels, and versioning for a Tauri desktop binary.

05 Quality Assurance Strategy

Testing pyramid adapted for local LLM/vision inference and offline data integrity.

06 Release Management

Packaging, auto-update, and rollback for macOS, Windows, and Linux.

07 Risk Register

Known technical risks with likelihood, impact, and mitigation.

08 Definition of Done

The bar every feature must clear before it ships.

01 — FOUNDATIONS

Purpose & Methodology

This document defines how AuraFit AI is built, tested, and shipped. Because the product is a single-binary desktop application with no backend service to operate, the process is optimized for short feedback loops, local reproducibility, and a very small team — rather than the coordination-heavy ceremonies of a multi-service cloud product.

Methodology: Agile–Kanban Hybrid

AuraFit AI is developed using a **continuous-flow Kanban board** layered with **two-week planning checkpoints** borrowed from Scrum. There are no fixed sprints with mandatory ceremonies; instead, work items flow through *Backlog* → *Design* → *In Progress* → *Verification* → *Done*, and the checkpoint is used only to re-prioritize the backlog against the phase goals in the Implementation Plan.

Why not full Scrum

A one-to-three person team building a single native app gets little value from sprint ceremonies; the overhead outweighs the coordination benefit.

Why not pure Waterfall

The vision-AI accuracy and prompt-engineering work is inherently exploratory and needs rapid iteration against real model output.

Why Kanban core

Continuous delivery of a desktop build lets every merged feature be manually smoke-tested the same day, keeping WIP visible and small.

GUIDING PRINCIPLE

Ship the local database and biometric engine first — they are dependency-free and testable in isolation — before layering in Ollama-dependent AI features that require an external runtime to be installed.

02 — PROCESS

Lifecycle Phases

Each release of AuraFit AI passes through seven phases. Unlike a typical SaaS SDLC, "Deployment" here means producing signed, auto-updating native installers rather than a server rollout, and "Operations" means telemetry-free crash reporting that the user opts into.

1

Ideation & Discovery

Validate feature scope against the four core pillars (biometrics, vision logging, meal generation, analytics). Confirm the local-AI approach remains feasible on target hardware (8-16GB RAM consumer laptops).

OUTPUT: Feature brief, model/hardware feasibility note

2

Requirements & Specification

Translate the brief into functional requirements, JSON/Zod schemas for any Ollama-facing contract, and SQLite schema deltas. This SDLC, the PRD, and the TRD are the primary artifacts reviewed at this gate.

OUTPUT: PRD updates, TRD updates, acceptance criteria

3

UI/UX Design

Screens are drafted in the shared shadcn/ui component library first, then validated against the design tokens in the UI/UX Brief before any React work begins.

OUTPUT: Approved screen mock, component checklist

4

Development

Rust/Tauri command handlers and React views are built in parallel behind a feature flag. Prompt templates for Ollama are versioned in source control alongside their Zod validators.

OUTPUT: Feature branch, passing local build

5

Verification & QA

Automated unit/integration tests plus a manual "golden set" pass — a fixed library of 40 reference meal photos re-run against the vision model whenever the prompt or model version changes.

OUTPUT: Test report, regression sign-off

6

Release & Deployment

Tauri bundles signed installers (.msi, .dmg, .ApplImage/.deb). The updater manifest is published; users on the previous version receive a delta-update prompt within 24 hours.

OUTPUT: Signed installers, changelog, GitHub release

7

Maintenance & Feedback

Opt-in, anonymized crash reports (no biometric or meal content ever included) and direct user feedback feed the next Ideation cycle.

OUTPUT: Triaged backlog items, patch releases

03 — PEOPLE

Roles & Responsibilities

AuraFit AI is designed to be buildable by a single full-stack engineer, but the responsibilities below stay separated even when one person wears every hat — each is a distinct review gate.

Role	Responsibility	Primary Artifacts
Product Owner	Owns feature scope, macro/goal logic accuracy, and dietary-guardrail correctness.	PRD, acceptance criteria
Rust/Backend Engineer	Tauri commands, SQLite access layer, Ollama REST integration, file I/O and image encoding.	TRD, Backend Schema
Frontend Engineer	React/TypeScript views, state management, Recharts visualizations, shadcn/ui composition.	UI/UX Brief, App Flow
AI/Prompt Engineer	Designs the JSON-schema prompts sent to llama3.2-vision and qwen2.5, tunes for portion-estimate accuracy.	TRD Vision Pipeline spec
QA / Test Owner	Maintains the golden-set photo library, cross-platform install testing, regression suite.	Test plan, golden-set results
Release Manager	Versioning, changelogs, code signing, updater manifest publication.	Release checklist

04 — INFRASTRUCTURE

Environments & Branching

There is no staging server — "environments" refer to build configurations of the same native binary.

DEV

Local Debug Build

Tauri dev server with hot reload, verbose logging, SQLite pointed at a disposable dev database file.

RC

Release Candidate

Optimized build with the golden-set test suite run against it before promotion; used for cross-OS smoke testing.

GA

General Availability

Signed, notarized/installer-verified binary distributed via GitHub Releases and the in-app updater.

Branch	Purpose	Merge Rule
main	Always-releasable trunk; tags map 1:1 to GA builds.	PR + passing CI + one review

feature/*	Individual features (e.g. feature/vision-logger).	Squash-merge into main
fix/*	Patch and hotfix branches cut from the latest tag.	Fast-tracked review

Semantic Versioning is used (MAJOR.MINOR.PATCH). MAJOR bumps only on breaking local-database schema changes that require a guided migration screen.

05 — QUALITY

Quality Assurance Strategy

Because two of the four core features depend on non-deterministic local LLM output, the QA pyramid adds a fourth, AI-specific layer on top of the conventional three.

Layer	Scope	Tooling
Unit	BMR/TDEE/macro math, Zod schema parsers, SQLite query builders.	Vitest (TS), cargo test (Rust)
Integration	Tauri command ↔ SQLite round-trips; Ollama request/response contract.	Vitest + Tauri test harness
End-to-End	Full user flows: onboarding → target generation → photo log → verification → save.	WebDriver/Tauri-driver
AI Accuracy (Golden Set)	40 reference meal photos with human-labeled ground truth, re-scored on every prompt or model change; tracks mean absolute % error on calories.	Custom Rust harness + CSV report

KNOWN LIMITATION

Vision-estimated portions carry inherent error (typically ±15–20% on calories for irregular home-plated meals). The Verification UI is a mandatory QA control, not an optional nicety — it is the primary mitigation, not the model.

06 — SHIPPING

Release Management

- Freeze & tag** — cut a release branch from main, run the full golden-set and cross-platform smoke suite.
- Build & sign** — `tauri build` produces platform bundles; Windows/macOS binaries are code-signed to avoid SmartScreen/Gatekeeper friction.
- Publish** — installers and the updater manifest (`latest.json`) are pushed to GitHub Releases.
- Rollout** — the Tauri updater plugin checks the manifest on launch; users can defer up to two versions before an update is required.
- Rollback** — a previous installer is always kept published; SQLite migrations are additive-only within a MINOR version so downgrades never corrupt local data.

07 — GOVERNANCE

Risk Register

Risk	Likelihood	Impact	Mitigation
Ollama not installed / models not pulled	Med	High	First-run setup wizard checks for the Ollama daemon and offers guided ollama pull commands with progress feedback.
Vision model misestimates portion weight	High	Med	Mandatory verification table before any log is committed; user can hand-correct grams and macros.
Malformed / non-JSON model output	Med	Med	Zod schema validation with one automatic re-prompt on failure, then graceful fallback to manual entry.
Consumer hardware too weak for 11B vision model	Med	High	Hardware check on first run; recommend quantized model variant or manual/barcode-free lookup path.
SQLite schema migration failure	Low	High	Automatic pre-migration backup copy of the .db file; additive-only migrations within a MINOR version.
Cross-platform UI inconsistency (Win/macOS/Linux)	Med	Low	shadcn/ui + Tailwind for consistent rendering; RC smoke test on all three OSes before GA.

08 — STANDARD

Definition of Done

A FEATURE IS NOT "DONE" UNTIL ALL OF THE FOLLOWING HOLD

- Code merged to main behind passing CI (unit + integration).
- No network call is made for any core function — verified via a network-monitor smoke test.
- Zod schemas exist for any Ollama-facing JSON contract, with a tested fallback path.
- Manual verification step exists for any AI-generated data before it is persisted.
- Feature is exercised on Windows, macOS, and Linux release-candidate builds.
- UI matches the design tokens defined in the UI/UX Brief.
- User-facing docs/changelog entry written.